SignLink – Final Implementation Report

Activity 3: Implementation Date: April 4, 2026 **Institution:** Ashesi University – Disability & Academic Support Services (DASS)

---

1. Project Overview

SignLink is a mobile application that connects deaf and hard-of-hearing students at Ashesi University with sign-language interpreters. The app enables students to request interpreter support for classes, events, and meetings, while allowing interpreters to manage their schedules, accept/decline requests, and communicate in real time. An admin role manages user accounts, assigns interpreters to requests, and creates campus events.

The application was built as a fully functional Flutter MVP with mock data services simulating backend behaviour, role-based navigation, and complete UI coverage of all 40+ designed screens.

---

2. Programming Language & Framework

Layer	Technology
Primary Language	Dart 3.10.7
UI Framework	Flutter 3.38.7 (Material 3)
Target Platforms	Android (APK), iOS (IPA)

---

3. Tools & IDE

Tool	Purpose
Visual Studio Code	Primary IDE
GitHub Copilot	AI-assisted code generation
Android SDK	Android build toolchain
Gradle	Android build system
Flutter SDK	Cross-platform UI framework
Dart Analyzer	Static analysis & linting
Git	Version control

---

4. Libraries & Packages

Package	Version	Purpose
`flutter`	SDK	Core UI framework
`provider`	^6.1.5	State management (ChangeNotifier)

`go_router`	^17.2.0	Declarative routing
`http`	^1.6.0	HTTP client for future API integration
`shared_preferences`	^2.5.5	Local key-value persistence (auth tokens, settings)
`image_picker`	^1.2.1	Camera & gallery access (timetable upload)
`intl`	^0.20.2	Date/time formatting & internationalisation
`cached_network_image`	^3.4.1	Efficient remote image loading
`flutter_animate`	^4.5.2	Entrance animations & transitions
`url_launcher`	^6.3.2	External links (email, phone, web)
`cupertino_icons`	^1.0.8	iOS-style icon set
`flutter_lints`	^6.0.0	Code quality linting rules

---

5. Architecture

The app follows a **clean, layered architecture** inspired by the Provider pattern:

```
lib/
├── main.dart                  # Entry point – MultiProvider bootstrap
├── app.dart                   # MaterialApp + named route table
├── config/
│   ├── app_routes.dart        # 40+ centralised route constants
│   └── app_theme.dart         # Material 3 ThemeData (brand colours, typography)
├── models/                    # Plain Dart data models (immutable)
│   ├── user_model.dart
│   ├── request_model.dart
│   ├── schedule_model.dart
│   ├── event_model.dart
│   ├── message_model.dart
│   └── notification_model.dart
├── services/                  # Data access layer
│   ├── mock_data_service.dart # In-memory mock data (students, interpreters, events…)
│   ├── auth_service.dart      # Login / sign-up simulation
│   ├── schedule_service.dart  # Schedule CRUD
│   ├── event_service.dart     # Events CRUD
│   └── chat_service.dart      # Messaging simulation
├── providers/                 # State management (ChangeNotifier)
│   ├── auth_provider.dart     # Auth state, current user, role
│   ├── schedule_provider.dart # Schedule & request state
│   ├── event_provider.dart    # Event list, sign-ups
│   └── chat_provider.dart     # Conversations & messages
├── widgets/                   # Reusable UI components
│   ├── primary_button.dart    # PrimaryButton, OutlineButton
│   ├── custom_app_bar.dart    # CustomAppBar, AppBarAction, UserAvatar
│   ├── info_card.dart         # InfoCard, StatCard, SectionHeader, StatusBadge
│   ├── event_tile.dart        # EventTile (card + progress bar + sign-up)
```

```
│   └── chat_bubble.dart       # ChatBubble (sent/received)
├── utils/
│   ├── constants.dart         # AppColors, AppSizes, AppStrings
│   ├── helpers.dart           # AppHelpers (date/time formatting, initials…)
│   └── validators.dart        # Form validators (email, password…)
└── screens/                   # All UI screens (see Section 6)
    ├── auth/
    ├── student/
    ├── interpreter/
    ├── admin/
    └── shared/
```

---

6. Screen Coverage

All 40+ designs from the implementation guide are fully implemented:

Authentication & Onboarding

Screen	File	Features
Splash Screen	`auth/splash_screen.dart`	Animated logo, 3-second auto-navigate, role-based redirect
Onboarding (3 slides)	`auth/onboarding_screen.dart`	PageView, animated dot indicators, skip / next / get-started
Login	`auth/login_screen.dart`	Email/password form, demo account quick-fill hints, role-based routing
Sign Up (Step 1 & 2)	`auth/sign_up_screen.dart`	Role chip (student / interpreter), personal details, T&C checkbox
Sign Up Success	`auth/sign_up_success_screen.dart`	Success animation → navigate to login
Forgot Password + Sent	`auth/forgot_password_screen.dart`	Email form + confirmation view (reset_password_sent)

Student Screens

Screen	File	Features
Dashboard 1 & 2	`student/student_dashboard.dart`	Greeting card, stats row, today's schedule, quick-actions grid, horizontal events scroll, bottom nav
Timetable	`student/student_timetable.dart`	TabController (Today / Upcoming / All), "Request Interpreter" CTA on each slot
Profile	`student/student_profile.dart`	Avatar, disability type badge, menu items, sign out
Events List	`student/events_list_screen.dart`	Search, tabs (Upcoming / Past / Registered), EventTile list

Event Detail	`student/event_detail_screen.dart`	SliverAppBar hero image, capacity progress bar, sign-up / already-registered state
Request Interpreter Flow (6 steps)	`student/request_interpreter_flow.dart`	Step indicator → request type → event details → date & time → review → success
Upload Timetable	`student/upload_timetable_screen.dart`	Drop-zone UI, camera/gallery picker, simulated upload progress

Interpreter Screens

Screen	File	Features
Dashboard	`interpreter/interpreter_dashboard.dart`	Stats row, today's assignments, quick actions, bottom nav
Schedule (All & 1)	`interpreter/interpreter_schedule.dart`	Tabs (All / Pending / Confirmed), Accept / Decline on pending items
Event Requests	`interpreter/event_requests_screen.dart`	Tabs (Pending / Approved / History), request cards with inline actions
Confirm Availability	`interpreter/confirm_availability_screen.dart`	Date range pickers, recurring toggle, day-of-week circle selector
Interpreter Profile	`interpreter/interpreter_profile.dart`	Profile header with green INTERPRETER badge, settings menu, sign out

Admin Screens

Screen	File	Features
Admin Dashboard	`admin/admin_dashboard.dart`	Gradient welcome card, stats (students / interpreters / pending), quick actions, recent requests list
Manage Users	`admin/manage_users_screen.dart`	Search bar, tabs (All / Students / Interpreters), popup menu (Edit / Suspend / Delete)
Assign Interpreter	`admin/assign_interpreter_screen.dart`	Request summary card, interpreter radio-selector cards, success view
Create Event	`admin/create_event_screen.dart`	Full form (title, description, location, capacity, date picker), success view

Shared Screens

Screen	File	Features
Messages List	`shared/messages_list_screen.dart`	Conversation list with online-dot, unread badge, time-ago
Chat	`shared/chat_screen.dart`	ChatBubble ListView (sent/received), video-call action, send bar

In-App Video Call	`shared/video_call_screen.dart`	Dark UI, picture-in-picture local video, mute/camera/speaker/end controls
Notifications	`shared/notifications_screen.dart`	Mark-all-read, type-coloured icons, unread dot indicators
Accessibility Settings	`shared/accessibility_settings_screen.dart`	Font-size slider with live preview, 4 toggle settings, reset button
Help & Support	`shared/help_support_screen.dart`	Contact DASS gradient card, 7-item expandable FAQ accordion

---

7. Data Layer

The current implementation uses a **mock data service** (`MockDataService`) that provides pre-seeded, in-memory data for:

- 3 demo users (student, interpreter, admin)
- 5 interpreter schedules
- 8 campus events
- 6 service requests
- 10 message/conversation records
- 8 notifications

All service files (`auth_service.dart` , `schedule_service.dart` , `event_service.dart` , `chat_service.dart`) are structured to be **drop-in replaceable** with real REST API calls using the bundled `http` package – no screen code changes would be required.

Persistence: `shared_preferences` is used for auth token and accessibility-settings storage across sessions.

---

8. State Management

Provider (ChangeNotifier) is used throughout:

Provider	State Managed
`AuthProvider`	Logged-in user, role, login/logout actions
`ScheduleProvider`	Class schedules, interpreter requests
`EventProvider`	Event list, sign-up status
`ChatProvider`	Conversations, messages, unread counts

All providers are mounted at app root via `MultiProvider` in `main.dart` , making state accessible from any screen without lifting state manually.

---

9. User Roles & Navigation

The app supports **three distinct roles**, each with its own navigation stack:

Role	Entry Screen	Key Permissions

Student	`student_dashboard`	Request interpreters, view events, upload timetable, message DASS
Interpreter	`interpreter_dashboard`	Accept/decline requests, manage schedule, set availability, message students
Admin (DASS Staff)	`admin_dashboard`	Assign interpreters, manage users, create events, view all requests

Role-based routing is handled in `splash_screen.dart` and `login_screen.dart` using `AuthProvider.currentUser.role`.

---

10. Design System

- **Primary colour:** `#003366` (Ashesi deep navy)
- **Accent:** `#0066CC` (action blue)
- **Success / Warning / Error:** `#16A34A` / `#F59E0B` / `#DC2626`
- **Typography:** Inter font family (system fallback)
- **Theme mode:** Light (Material 3 `colorScheme` seeded from primary)
- **Components:** `PrimaryButton`, `OutlineButton`, `StatCard`, `InfoCard`, `SectionHeader`, `StatusBadge`, `UserAvatar`, `EventTile`, `ChatBubble`

---

11. Build Output

Artefact	Path	Size
Debug APK	`signlink_app/build/app/outputs/flutter-apk/app-debug.apk`	~141 MB

Build command used:

```
flutter build apk --debug
```

To install on an Android device:

```
flutter install
# or
adb install build/app/outputs/flutter-apk/app-debug.apk
```

Demo login credentials (tap to auto-fill in app):

Role	Email	Password
Student	`alex.johnson@ashesi.edu.gh`	`Student@123`
Interpreter	`kofi.mensah@ashesi.edu.gh`	`Interp@123`
Admin	`sarah.asante@ashesi.edu.gh`	`Admin@123`

---

12. Screen Count Summary

Category	HTML Designs	Flutter Screens
Authentication	8	6 files (some consolidate 2 designs)
Student	9	7 files
Interpreter	5	5 files
Admin	4	4 files
Shared	6	6 files
Total	**32+**	**28 files covering all 40+ design states**

---

13. Known Limitations (MVP Scope)

1. **No live backend** – all data is mock/in-memory; resets on app restart.
2. **No push notifications** – notification data is static mock.
3. **No real video calling** – video call screen is a UI prototype only (no WebRTC/Agora integration).
4. **No real file upload** – timetable upload simulates progress without cloud storage.
5. **No offline caching** – network-dependent features would need connectivity.

These are intentional MVP boundaries; the service layer is architected for straightforward replacement with live APIs.

Report generated: April 4, 2026 | Flutter 3.38.7 | Dart 3.10.7 | signlink_app v1.0.0+1